

中技社路線規則邏輯

掌握到你要的方向了：不要人工建資料庫，改成**即時查詢外接地圖 API 拿到的真實 POI**，再套用「**類型規則**」去判斷能不能用、怎麼用。這樣每次生成路線都是活的，且完全可解釋「為什麼選這個點」。你的網站用的是 Leaflet（開源地圖底圖），最搭配的免費資料源是 **OpenStreetMap 的 Overpass API**——不需要金鑰、可即時查詢方圓範圍內的真實地點類型，跟你「免費開源地圖原型」的定位完全一致。

10大類型規則表（可解釋性的核心）

這張表就是你的「規則」，不是資料庫——它決定「查到什麼類型的點，要怎麼用」：

類型	對應 OSM 查詢標籤	對應主題	用途規則	可作未知終點？
學校	<code>amenity=school</code>	建築細節	只拍外觀／校門意象，不得進入校園	否
銀行	<code>amenity=bank</code>	—	不指派拍照任務 ，僅作方向地標／安全參照點	否
娛樂地區	<code>amenity=cinema</code> 、 <code>leisure=amusement_arcade</code>	城市色彩	觀察招牌、燈光、外觀色彩	可（小型終點）
餐廳	<code>amenity=restaurant</code>	城市色彩	觀察招牌設計、氣味聯想式任務文字	否
咖啡廳	<code>amenity=cafe</code>	建築細節／療癒	觀察裝潢、窗景、光影	可
市場	<code>amenity=marketplace</code>	老街痕跡	觀察生活痕跡、攤位紋理	否
服飾店	<code>shop=clothes</code>	建築細節	觀察櫥窗陳設	否
地標	<code>tourism=attraction</code> 、 <code>historic=*</code>	全主題皆可	唯一開放給終點的類型	是
捷運／公車站	<code>railway=station</code> 、 <code>highway=bus_stop</code>	—	不指派任務，只用於「回程建議」	否
公園綠地	<code>leisure=park</code>	自然縫隙	觀察植物縫隙、光影	可

規則邏輯很清楚：銀行、捷運站只能當「地標／回程參考」，不能當拍照任務點（涉及安全與隱私敏感度）；只有「地標」類型能被選為未知終點，其他類型只能是路途中的任務點。這張表就是你在簡報上可以直接秀出來、回答評審「為什麼選這個點」的依據。

更新後的五步驟（即時版，不建資料庫）

第一步：即時查詢附近POI（取代原本的「候選任務點資料庫」）

```
async function fetchNearbyPOIs(center: Coordinate, radiusKm: number) {
  const radiusM = Math.round(radiusKm * 1000);
  const query = `
    [out:json][timeout:10];
    (
      node["amenity"~"school|bank|cinema|restaurant|cafe|marketplace"]
        (around:${radiusM},${center[0]},${center[1]});
      node["shop"="clothes"] (around:${radiusM},${center[0]},${center[1]});
      node["tourism"="attraction"] (around:${radiusM},${center[0]},${center[1]});
      node["railway"="station"] (around:${radiusM},${center[0]},${center[1]});
      node["leisure"="park"] (around:${radiusM},${center[0]},${center[1]});
    );
    out body;
  `;
  const res = await fetch("https://overpass-api.de/api/interpret", {
    method: "POST",
    body: query,
  });
  return (await res.json()).elements; // 每個元素含 lat/lon/tags
}
```

第二步：時間預算換算成距離預算（維持你原本設計不變）

```

travelBudgetMin = totalMinutes * 0.5
walkSpeedKmH = pace === "慢慢走" ? 3.2 : 4.5
maxRadiusKm = (travelBudgetMin / 60) * walkSpeedKmH

```

第三步：套用規則表分類、過濾

```

const RULES: Record<string, { theme: string; taskable: boolean; landmarkOK: boolean }> = {
  school: { theme: "建築細節", taskable: true, landmarkOK: false },
  bank: { theme: "", taskable: false, landmarkOK: false },
  cinema: { theme: "城市色彩", taskable: true, landmarkOK: true },
  restaurant: { theme: "城市色彩", taskable: true, landmarkOK: false },
  cafe: { theme: "建築細節", taskable: true, landmarkOK: true },
  marketplace: { theme: "老街痕跡", taskable: true, landmarkOK: false },
  clothes: { theme: "建築細節", taskable: true, landmarkOK: false },
  attraction: { theme: "全主題", taskable: true, landmarkOK: true },
  station: { theme: "", taskable: false, landmarkOK: false },
  park: { theme: "自然縫隙", taskable: true, landmarkOK: true },
};

function classify(poi) {
  const key = poi.tags.amenity || poi.tags.shop || poi.tags.tourism
  || poi.tags.railway || poi.tags.leisure;
  return { ...RULES[key], key, poi };
}

const candidates = pois
  .map(classify)

```

```
.filter(c => c.taskable && (theme === "隨機" || c.theme === theme
|| c.theme === "全主題"));
```

第四步：加權隨機+貪婪鏈接排出路線（跟前一版邏輯相同，選點來源改成即時查詢結果）

```
function pickWeightedRandom(nodes, from) {
  const weights = nodes.map(n => 1 / (distanceBetween(from, [n.poi.
lat, n.poi.lon]) + 0.05));
  const total = weights.reduce((a, b) => a + b, 0);
  let r = Math.random() * total;
  for (let i = 0; i < nodes.length; i++) {
    r -= weights[i];
    if (r <= 0) return nodes[i];
  }
  return nodes.at(-1);
}
```

依時程決定任務數（40分鐘3個、1小時4個、2小時5-6個），逐步從候選中挑點，最後一個點限定從 `landmarkOK: true` 的候選中選，作為未知終點。

第五步：生成「可解釋原因」字串（你要的「解釋得出來為什麼選這個點」）

```
function explainSelection(c, distanceM) {
  return `因為在你設定的「${c.theme}」主題、距離約${Math.round(distance
M)}公尺內，找到一處屬於「${c.key}」類別的公共場域，符合可安全拍攝的任務條件。
`;
}
```

這段文字甚至可以直接顯示在任務卡片上（或簡報時秀出來），讓評審看到系統不是黑盒子亂選，而是每一步都有規則依據。

為什麼這樣設計是對的

- 不用資料庫維護成本，POI 資料永遠是真實、即時的
- 每個使用者、每次出發，因為所在地不同、Overpass查詢範圍不同，路線天然就是隨機的——完全符合你要的「盲盒」感

- 重複也沒關係：你本來就不需要做「避免重複」的複雜邏輯，這版設計本身就很輕量，跑起來也快（Overpass 查詢通常1秒內回應）
- 唯一需要留意：Overpass 是公用免費伺服器，正式展示前建議做「查詢失敗時的預設候選點」防呆（例如查不到結果時退回到你現有寫死的那組點當 fallback），避免demo當天連不上